EMOPLOYEE MANAGEMENT SYSTEM

A FULL STACK PROJECT REPORT

Submitted by

IMMANUVEL R

(22ITR035)

KRITTIKA R

(22IT053)

BARATH KUMAR R

(22ITL121)

in partial fulfilment of the requirements

for the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY



KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638 060

NOVEMBER 2024

DEPARTMENT OF INFORMATION TECHNOLOGY

KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638060

NOVEMBER 2024

BONAFIED CERTIFICATE

This is to certify that the Project report entitled **EMOPLOYEE MANAGEMENT SYSTEM** is the Bonafide record of project work done by **IMMANUVEL R(22ITR035), KRITTIKA R (22ITR053)** and **BARATH KUMAR R (22ITL121)** in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology in **INFORMATION TECHNOLOGY** of Anna university, Chennai during the year 2024-2025.

SUPERVISOR **HEAD OF THE DEPARTMENT**

(Signature with seal)

Date:

Submitted for the end semester viva voce examination held on _____.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DEPARTMENT OF INFORMATION TECHNOLOGY

KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638060

NOVEMBER 2024

DECLARATION

We affirm that the Project Report titled **EMOPLOYEE MANAGEMENT SYSTEM** being submitted in partial fulfilment of the requirements for the award of Bachelor of Technology is the original work carried out by us. It has not formed the part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Date:

IMMANUVEL R
(22ITR035)

KRITTIKA
(22ITR089)

BARATH KUMAR R
(22ITL121)

I certify that the declaration made by the above candidates is true to the best of my knowledge.

Date:

Name and Signature of the Supervisor with seal

ABSTRACT

In today's dynamic business environment, efficient management of human resources is crucial for organizational success. Many companies struggle with outdated, manual processes for employee data management, attendance tracking, leave management, and payroll calculation. These inefficiencies lead to increased administrative overhead, data inconsistencies, and reduced productivity.

The Employee Management System (EMS) addresses these challenges by providing a comprehensive, web-based solution for streamlining workforce administration. This project aims to develop an intuitive, secure, and scalable platform that automates key HR processes.

The EMS incorporates several key features like centralized storage, Attendance Tracking for automated recording, Leave Management for leave applications, approvals, and tracking, payroll Calculation for automated salary computation considering attendance, leaves, and deductions. The system is built using modern web technologies, including React for the frontend, Node.js and Express for the backend, and MongoDB for data storage.

By implementing the EMS, organizations can expect significant improvements in HR process efficiency, reduced administrative workload, enhanced data accuracy, and improved employee satisfaction. The system's user-friendly interface and mobile responsiveness enable easy access for both HR personnel and employees, fostering better communication and transparency.

ACKNOWLEDGEMENT

First and foremost, we acknowledge the abundant grace and presence of Almighty throughout different phases of the project and its successful completion.

We wish to express our gratefulness to our beloved Correspondent **Thiru.A.K.ILANGO B.Com., M.B.A., LLB.,** and all the trust members of Kongu Vellalar Institute of Technology Trust for providing all the necessary facilities to complete the project successfully.

We express our deep sense of gratitude to our beloved Principal **Dr.V.BALUSAMY B.E.(Hons)., M.Tech., Ph.D.,** for providing us an opportunity to complete the project.

We express our gratitude to **Dr. S. ANANDAMURUGAN M.E., Ph.D.,** Head of the Department, Department of Information Technology for his valuable suggestions.

We are thankful to our Project Coordinators **Dr. G.K. Kamalam BE., ME., PhD.,** and **Ms. R. Sandhiya BE., ME.,** for the valuable guidance and support to complete our project successfully.

We are highly indebted to **Dr. S. ANANDAMURUGAN M.E., Ph.D.,** Department of Information Technology for his valuable supervision and advice for the fruitful completion of the project.

We are thankful to the faculty members of the Department of Information Technology for their valuable guidance and support.

TABLE OF CONTENTS

CHAPTER No	TITLE	PAGE No
	ABSTRACT	**iv**
	LIST OF FIGURES	**viii**
	LIST OF ABBREVIATIONS	**ix**
1.	**INTRODUCTION**	**1**
	1.1 INTRODUCTION	1
	1.2 OBJECTIVE	1
2.	**SYSTEM SPECIFICATION**	**2**
	2.1 HARDWARE REQUIREMENTS	2
	2.2 SOFTWARE REQUIREMENTS	2
	2.3 SOFTWARE DESCRIPTION	3
	2.3.1 Visual Studio Code	3
	2.3.2 NodeJS	3
	2.3.3 MongoDB	5
	2.3.4 ReactJS	5
3.	**SYSTEM DESIGN**	**7**
	3.1 USE CASE DIAGRAM	7
	3.2 CLASS DIAGRAM	8
	3.3 SEQUENCE DIAGRAM	9
	3.4 ACTIVITY DIAGRAM	10

3.5 DATABASE DESIGN 11

3.6 MODULES DESCRIPTION 12

3.6.1 Authentication Module 12

3.6.2 Employee Information Module 12

3.6.3 Attendance Tracking Module 13

3.6.4 Leave Management Module 14

3.6.5 Payroll Calculation Module 14

4. SYSTEM TESTING 14

4.1 SYSTEM TESTING 14

4.2 UNIT TESTING 14

4.3 MODULE TESTING 15

4.4 INTEGRATION TESTING 15

4.5 VALIDATION TESTING 15

RESULTS 17

CONCLUSION AND FUTURE WORK 18

APPENDIX 1- CODING 19

APPENDIX 2- SNAPSHOTS 48

REFERENCES 52

LIST OF FIGURES

FIGURE No.	FIGURE NAME	PAGE No.
3.1	USE CASE DIAGRAM	7
3.2	CLASS DIAGRAM	8
3.3	SEQUENCE DIAGRAM	9
3.4	ACTIVITY DIAGRAM	10
A2.1	LANDING PAGE	47
A2.2	ADMIN PANEL	47
A2.3	EMPLOYEE PANEL	48
A2.4	ATTENDANCE PAGE	48
A2.5	SALARY PAGE	49
A2.6	LEAVE PAGE	49
A2.7	COMMON MESSAGE PAGE	50
A2.8	LEAVE APPLICATION PAGE	50
A2.6	LEAVE PAGE	49

LIST OF ABBREVIATIONS

CSS	Cascading Style Sheets
HTML	HyperText Markup Language
ID	Identification
JS	JavaScript
JSX	JavaScript XML
NFC	Near-Field Communication
QR	Quick Response
URL	Uniform Resource Locator

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

The Employee Management System (EMS) is a comprehensive web-based solution designed to streamline and automate various aspects of human resource management within organizations. In today's fast-paced business environment, efficient management of human resources is crucial for organizational success. Many companies still rely on outdated, manual processes for employee data management, attendance tracking, leave management, and payroll calculation. These inefficiencies often lead to increased administrative overhead, data inconsistencies, and reduced productivity. The EMS addresses these challenges by providing a centralized platform that integrates key HR functions, enhancing overall organizational efficiency and employee satisfaction.

1.2 OBJECTIVE

The project aims to develop a user-friendly, secure, and scalable web-based system for managing employee information and HR processes which automate and streamline key HR functions such as employee data management, attendance tracking, leave management, and payroll calculation. And also to improve data accuracy and consistency across various HR processes by enhanc e communication and transparency between employees and HR personnel. Also provide real-time access to employee information and analytics for informed decision-making. The objective extends to reduce administrative workload and improve overall HR process efficiency and ensure compliance with data protection regulations and implement robust security measures.

CHAPTER 2

SYSTEM SPECIFICATION

2.1 HARDWARE SPECIFICATION

Processor	:	Intel Pentium
Processor Speed	:	250MHz to 667MHz
RAM	:	4GB RAM
Hard Disk	:	256GB
Keyboard	:	Standard 104 enhanced
Mouse	:	Local PS/2

2.2 SOFTWARE REQUIREMENTS

Platform	:	Visual Studio Code
Server-Side Script	:	NodeJS
Database	:	MongoDB
Library	:	ReactJS

2.3 SOFTWARE DESCRIPTION

2.3.1 Visual Studio Code

Visual Studio Code is a versatile and user-friendly code editor used by developers worldwide. It supports a wide range of programming languages and frameworks, including JavaScript, React, and Node.js. For this project, VS Code provides a convenient environment for writing, editing, and debugging code related to the server-side scripting with Node.js, as well as client-side scripting with React.js. Its built-in terminal and debugging tools streamline the development process, while its customizable features allow developers to tailor the editor to their specific needs, enhancing productivity and efficiency throughout the project lifecycle.

ES6+ React/Redux/React-Native

ES6 React/Redux/React-Native snippets provide code templates designed to simplify and accelerate the development process for building applications with React, Redux, and React Native. These snippets utilize the powerful features introduced in ES6, allowing developers to quickly scaffold common patterns like React components, Redux actions, reducers, and React Native components. By incorporating ES6 syntax, such as arrow functions, destructuring, template literals, and modules, these snippets make it easier to write clean and concise code. This collection of snippets supports the creation of both web applications in React and cross-platform mobile applications in React Native, while simplifying state management in Redux. Whether developing user interfaces, managing state, or working on cross-platform features, these ES6 snippets help developers stay focused on creating features and solving problems instead of repeating boilerplate setup.

2.3.2 NodeJS

Node.js is a powerful server-side JavaScript runtime environment commonly used for building scalable and efficient web applications. In our project, Node.js serves as the server-side scripting language, handling tasks such as routing, handling HTTP requests, and interacting with the MongoDB database. Its event-driven architecture and non-blocking I/O

operations make it well-suited for handling concurrent connections and processing requestsefficiently. With its vast ecosystem of libraries and frameworks, Node.js allows for rapid development and deployment of server-side applications. Additionally, its compatibility with Visual Studio Code provides a seamless development experience, enabling developers to write, test, and debug server-side code effectively within a unified environment. Overall, Node.js plays a crucial role in this project by powering the backend infrastructure and facilitating the seamless integration of frontend and backend components.

1. **Bcryptjs**

Bcryptjs is a library used for hashing passwords in JavaScript. It provides a secure way to hash passwords before storing them in a database or comparing them during authentication processes. By using bcryptjs, developers can ensure that user passwords are encrypted and protected against common security threats like brute force attacks and rainbow table attacks.

2. **Express**

Express is a minimalist web application framework for Node.js. It provides a robust set of features for building web servers and APIs, including routing, middleware support, template engines, and HTTP utilities. With its simple and flexible design, Express enables developers to quickly create scalable and maintainable web applications.

3. **Jsonwebtoken**

JSON Web Token is an open standard (RFC 7519) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object. JWTs are commonly used for authentication and authorization in web applications. The jsonwebtoken library provides utilities for generating, parsing, and verifying JWTs, making it easy to implement token-based authentication and authorization systems.

4. **Nodemon**

Nodemon is a utility that monitors for changes in Node.js applications and automatically restarts the server when changes are detected. It eliminates the need to manually stop and restart the server every time a code change is made, improving developer productivity and

workflow efficiency. Nodemon supports features like watching for file changes, ignoring specific files or directories and running custom scripts before and after restarting the server, making it an essential tool for Node.js development.

2.3.3 MongoDB

MongoDB is a flexible and scalable NoSQL database platform ideal for managing complex data structures, making it invaluable for our project's ecommerce website. With MongoDB's document-oriented architecture, we can efficiently store and manage diverse types of data relevant to our ecommerce platform, such as user information, appointment details, and messages. Its schema-less nature allows for dynamic and evolving data models, accommodating the dynamic nature of ecommerce data. For this project, MongoDB's scalability is crucial as it can seamlessly handle increasing volumes of data and traffic, ensuring optimal performance even during peak usage periods. This scalability ensures that the website remains responsive and can accommodate growth without sacrificing performance. MongoDB's powerful query language and indexing capabilities enable us to perform complex queries efficiently, facilitating tasks such as retrieving user information, appointment details, and message history. Overall, MongoDB provides a robust and scalable e database solution that empowers us to create a secure, efficient, and highly performant ecommerce website. Its flexibility, scalability, and powerful features make it an indispensable component of our project, enabling us to meet the diverse data management needs of our ecommerce platform effectively.

2.3.4 ReactJS

React, a widely-used JavaScript library, plays a crucial role in our ecommerce project. Its component-based architecture facilitates the creation of reusable UI elements. React's virtual DOM optimizes performance by minimizing full page reloads, ensuring a seamless user experience even under high traffic. Furthermore, React's extensive community support provides access to a plethora of third-party libraries and tools, streamlining the development process and enhancing productivity. Overall, React significantly contributes to improving user experience and efficiency in our ecommerce project.

1. **axios:**

Axios is a promise-based HTTP client for making HTTP requests in browser-based and Node.js applications. It provides a simple and intuitive API for performing asynchronous HTTP requests, handling request and response data, and intercepting request and response lifecycle events. Axios supports features like request and response interception, request cancellation, automatic JSON data parsing, and more, making it a popular choice for handling AJAX requests in JavaScript applications.

2. **react-dom:**

React DOM is a package that serves as the entry point for working with the DOM in React applications. It provides methods for rendering React components into the DOM, updating component state, handling events, and interacting with the browser's Document Object Model (DOM). React DOM is essential for building React applications that interact with the user interface and manipulate the DOM based on user interactions and data changes.

3. **react-redux:**

React Redux is the official React bindings for Redux, a predictable state container for JavaScript applications. It provides a set of APIs and components that simplify the process of integrating Redux with React applications, including <Provider> component for passing the Redux store to React components, connect() function for connecting React components to the Redux store, and <Selector> component for selecting data from the Redux store. React Redux enables developers to manage application state more efficiently and maintain application consistency across components.

4. **react-router-dom:**

React Router DOM is a package that provides routing for React applications. It allows developers to define routes and navigation logic using components and props, making it easy to create single-page applications with multiple views. React Router DOM provides a simple API for defining route configurations, handling navigation events, and passing parameters, making it a popular choice for managing client-side routing in React applications.

CHAPTER 3

SYSTEM DESIGN

3.1 USE CASE DESIGN

A use case diagram is a dynamic or behaviour diagram in UML (Unified Modelling Language).Use case diagrams model the functionality of a system using actors and use cases. Use cases are a set of actions and functions that the system needs to perform. A "system" is something being developed or operated, here a website. The "actors" are people or entities operating under defined roles within the system as shown below in Figure 3.1.

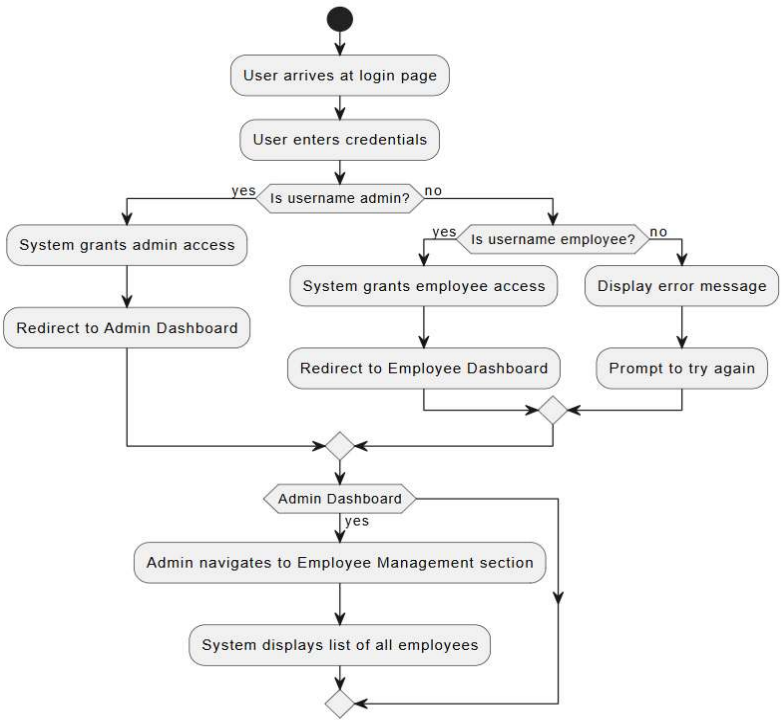


Fig 3.1 Use Case Diagram

3.2 CLASS DIAGRAM

A class diagram is a static structure diagram in UML (Unified Modeling Language) that illustrates the structure of a system by showing the classes of the system, their attributes, methods, and relationships between them. Classes represent the blueprint for objects, encapsulating data and behavior. Attributes are the data members of a class, while methods represent the operations that can be performed on the class's objects.

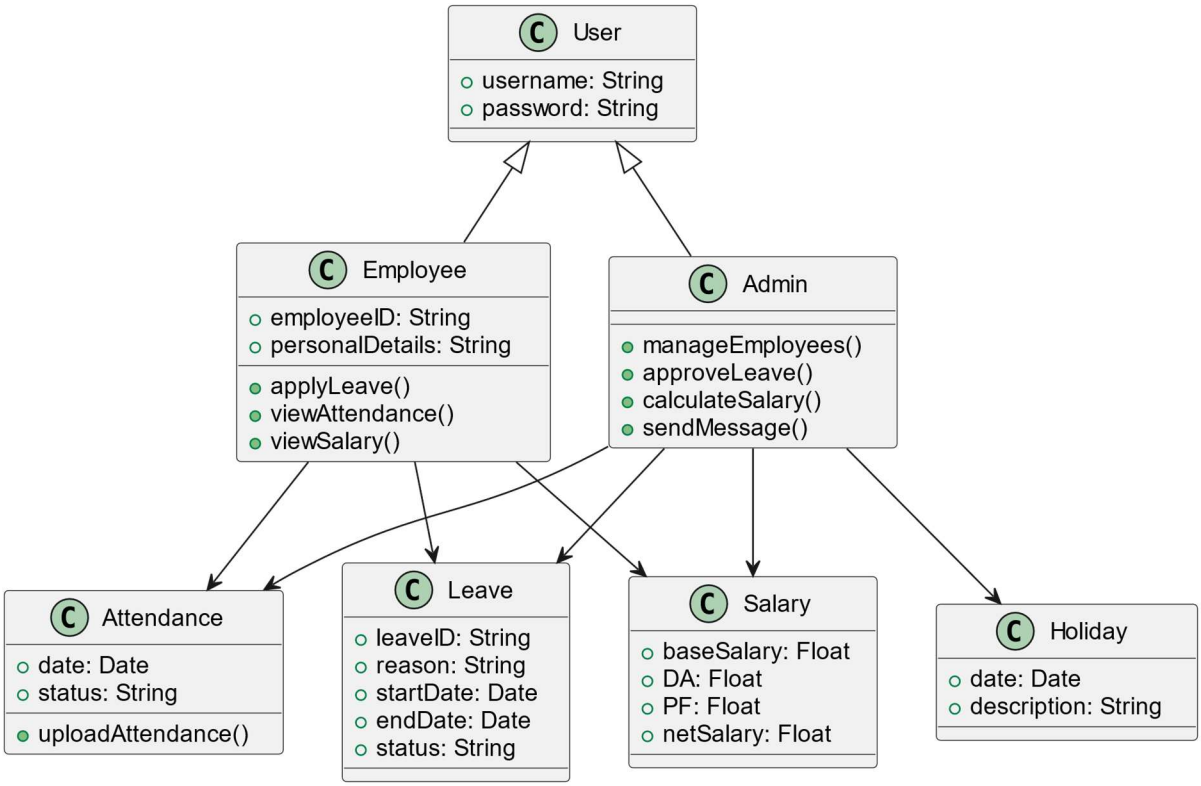


Fig 3.2 Class Diagram

3.3 SEQUENCE DIAGRAM

A sequence diagram is a form of interaction diagram which shows objects as lifelines running down the page, with their interactions over time represented as messages drawn as arrows from the source lifeline to the target lifeline. Sequence diagrams are good at showing which objects communicate with which other objects and what messages trigger those communications as shown below in Figure 3.3. Sequence diagrams are not intended for showing complex procedural logic.

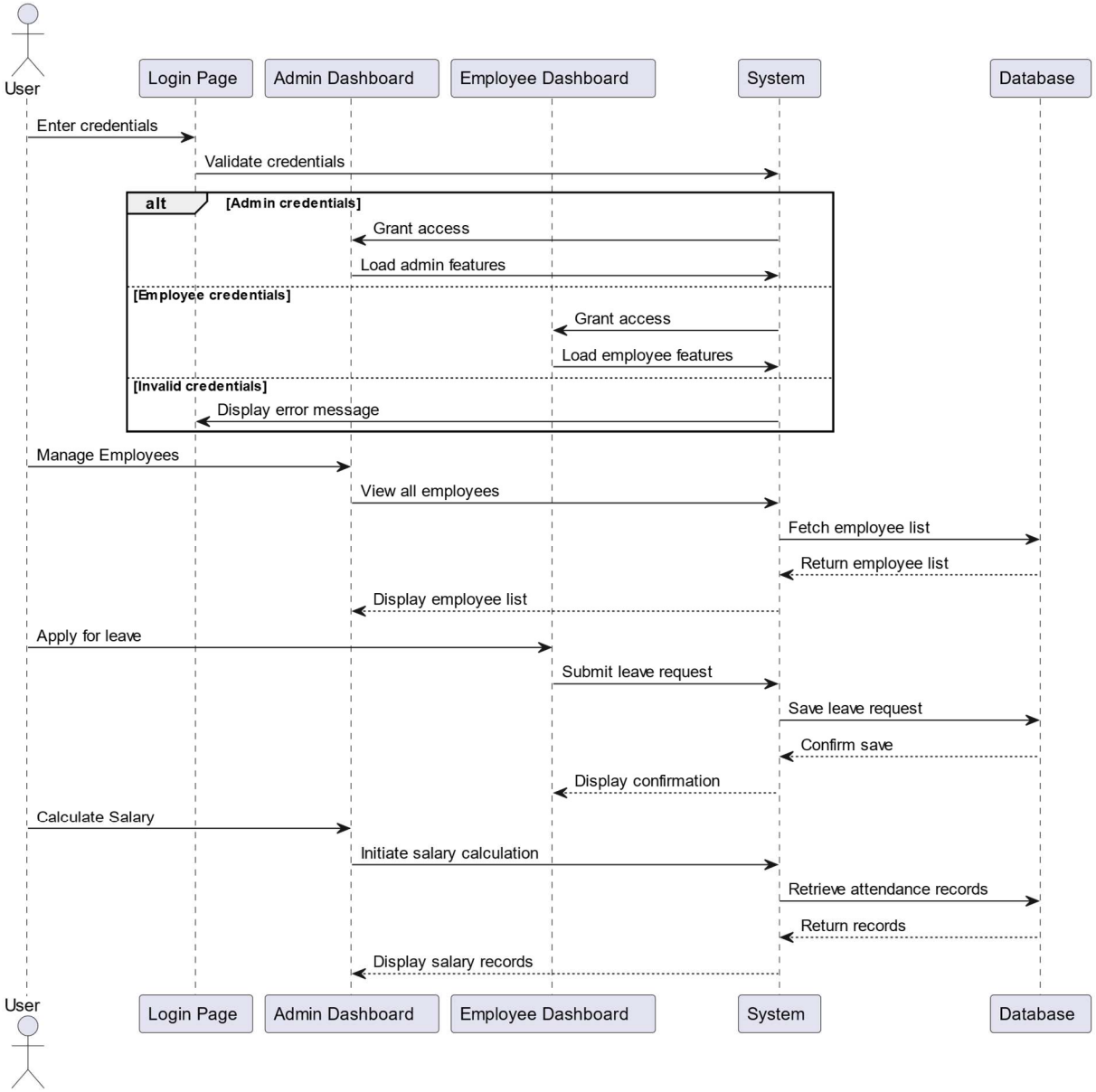


Fig 3.3 Sequence Diagram

3.4. ACTIVITY DIAGRAM

In UML, the activity diagram is used to demonstrate the flow of control within the system rather than the implementation. It models the concurrent and sequential activities. The activity diagram helps in envisioning the workflow from one activity to another. It put emphasis on the condition of flow and the order in which it occurs. The flow can be sequential, branched or concurrent and to deal with such kinds of flows, the activity diagram has come up with a fork, join, etc. as shown in the Figure 3.4 below.

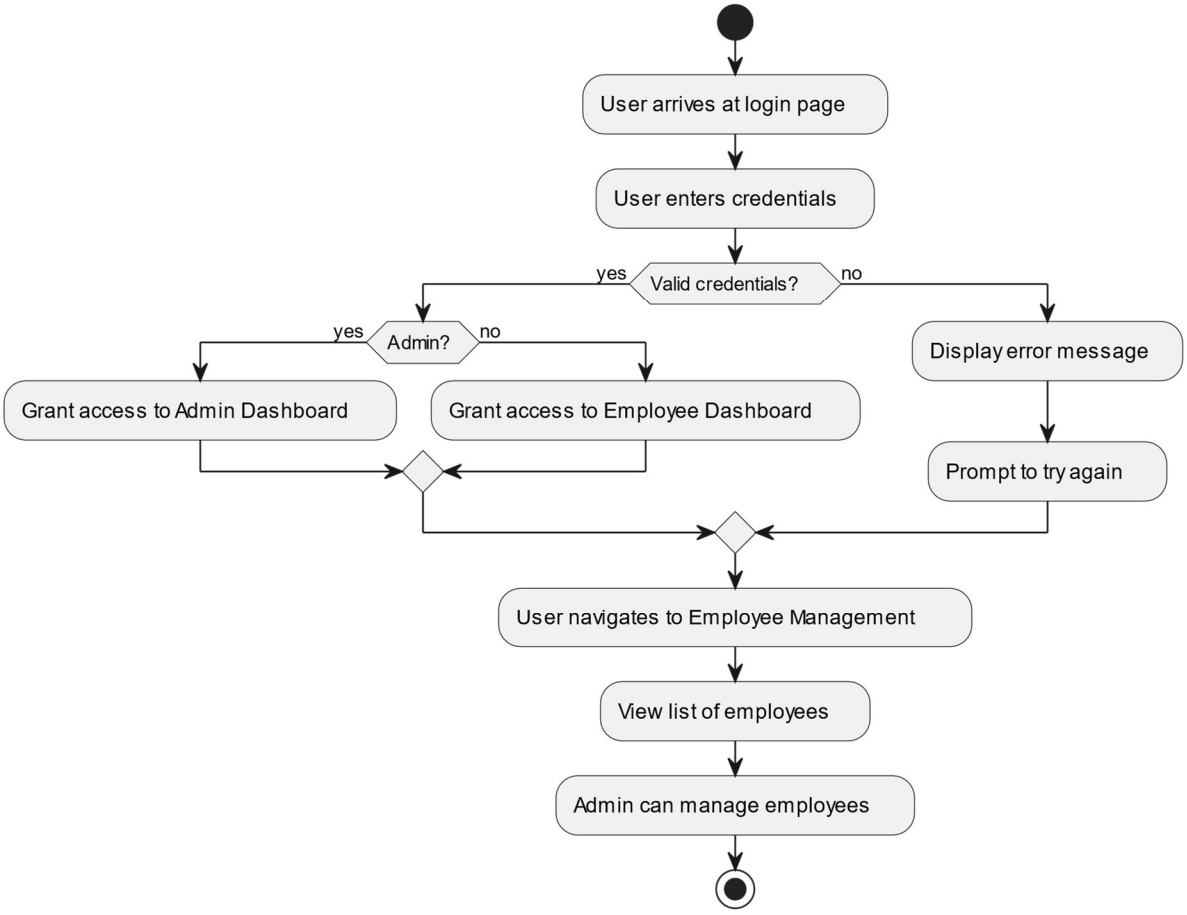


Fig 3.4 Activity Diagram

3.5 DATABASE DESIGN

3.5.1 USER SCHEMA

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({

  email: String,

  password: String,

  role: String,

});

module.exports = mongoose.model('User', userSchema);
```

3.5.2 EMPLOYEE SCHEMA

```
const employeeSchema = new mongoose.Schema({
  employeeId: { type: String, required: true, unique: true },
  name: { type: String, required: true },
  email: { type: String, required: true },
  department: { type: String, required: true },
  salary: { type: Number, required: true },
  phoneNumber: { type: String, required: true },
  sex: { type: String, required: true },
  qualifications: { type: String, required: true },
  role: { type: String, required: true },
  dob: { type: String, required: true },
  joiningDate: { type: String, required: true },
  resignationDate: { type: String },
```

```
  experience: { type: String, required: true },
  experiencedRole: { type: String },
  attendance: [{ date: String, status: String }],
  pf: { type: Number, required: true },
  da: { type: Number, required: true },
  monthlySalary: {
    month: String,
    presentDays: Number,
    totalDays: Number,
    netSalary: Number
  }
});

// Employee Model
const Employee = mongoose.model('Employee', employeeSchema);
```

3.6 MODULES DESCRIPTION

3.6.1 AUTHENTICATION MODULE

This module manages user access, roles, and permissions within the system. It ensures that sensitive employee data is protected and only accessible to authorized personnel. This module includes features for user authentication, password management, activity logging, and audit trails. It also allows system administrators to configure system settings, manage master data, and perform database backups and restorations.

3.6.2 EMPLOYEE INFORMATION MANAGEMENT MODULE

This module serves as the central repository for all employee data. It allows HR personnel to add, view, edit, and manage comprehensive employee profiles. The module stores essential information such as personal details, contact information, job roles, departments,

qualifications, and employment history. It also includes features for document management, allowing storage and retrieval of important employee documents.

3.6.3 ATTENDANCE MANAGEMENT MODULE

The Attendance Management Module automates the process of tracking employee attendance. It supports various methods of attendance marking, including manual entry, file upload, and integration with biometric systems. The module provides real-time visibility of employee attendance, generates daily and monthly attendance reports, and flags irregularities for HR review. It also calculates working hours, overtime, and maintains a log of employee time-in and time-out records.

3.6.4 LEAVE MANAGEMENT MODULE

This module streamlines the entire leave management process. It allows employees to submit leave requests online, specifying leave types, dates, and reasons. The module routes these requests to appropriate managers for approval, sends notifications, and updates leave balances automatically. It provides a calendar view of team leaves, helps in managing different leave types, and generates leave reports for HR analysis.

3.6.5 PAYROLL MANAGEMENT MODULE

The Payroll Management Module automates salary calculations based on attendance data, leave records, and predefined salary structures. It handles various components of salary including basic pay, allowances, deductions, and taxes. The module generates pay slips, facilitates bulk salary processing, and maintains a history of all payroll transactions. It also supports integration with financial systems for seamless salary disbursement.

CHAPTER 4

SYSTEM TESTING

4.1 SYSTEM TESTING

The goal of the software business worldwide has always been to offer software products of the highest quality with distinctive characteristics. The team however, cannot ensure these aspects without testing software components under a variety of anticipated and unforeseen circumstances. The process of detecting flaws in a developed product is called software testing. Additionally, it helps in the detection of flaws, gaps and missing requirements by determining whether the actual findings can be reconciled with the anticipated outcomes.

The last step before a product is introduced to the market is testing. It involves looking at, analyzing, observing, and rating several features of a product. Software testing is crucial because it allows any faults or errors in the software to be found early and fixed before the software product is delivered. Reliability, security and high performance are all ensured by thoroughly tested software, which also leads to saving time, cost effectiveness, and customer pleasure. Four phases are included in system testing.

- Unit testing
- Module testing
- Integration testing
- Validation testing

4.2 UNIT TESTING

The smallest chunk of code that can be logically separated in a system is called a unit, and a unit test is a method of testing a unit. The developer performs this testing concurrently with the development process, that is, adjacent to coding. To test smaller units as they are constructed, the user may utilize several temporary functions, methods, or even simply lines of code inside the main code. This provides a much more detailed understanding of how the

code is operating.

4.3 MODULE TESTING

Module testing for the Digital Contact Manager involves systematically verifying individual components, such as the Login page, Profile page, QR Code page, and Admin dashboard, to ensure they function as intended and meet the requirements. Each module is tested independently. For the Login page, testing focuses on user authentication, error handling, and security measures to ensure secure access. The Profile page is tested to verify that users can input, update, and save contact information accurately, with real-time reflection on the unique URL. The QR Code page is checked to ensure proper QR generation, scanning functionality, and compatibility across devices. By conducting thorough module testing, any potential issues are promptly addressed, ensuring that the Digital Contact Manager allows usersto manage and share their contact information efficiently and securely.

4.4 INTEGRATION TESTING

Integration testing for the Digital Contact Manager involves testing the combined functionality of all modules, including the Login page, Profile page, QR Code page, and Admin dashboard, to ensure seamless operation when integrated. The primary objective is to validate the interactions and data flow between these modules, confirming that they function together smoothly. During integration testing, the system is evaluated to verify that data entered on the Profile page updates in real-time on the unique URL and that the QR Code page accurately reflects the latest information. Upon successful completion of integration testing without any detected defects, the application is deemed to meet all requirements, ensuring that the Digital Contact Manager provides users with a cohesive, efficient experience for managing and sharing contact information.

4.5 VALIDATION TESTING

Validation testing for the Digital Contact Manager involves evaluating whether the

system meets the intended requirements and user expectations. This testing phase ensures that the platform performs correctly in real-world scenarios, addressing the specific objectives. During validation testing, effective input validation is applied across all forms, including login, profile update, and contact information fields, to ensure data integrity and security. The primary goal is to confirm that the Digital Contact Manager functions as expected from a user's perspective, providing a smooth and reliable experience when managing and sharing contact details. By verifying the system's behavior and usability, validation testing ensures that the platform meets the business requirements and delivers value to users.

CHAPTER 5

RESULTS

The implementation of the Employee Management System (EMS) has yielded significant positive outcomes for organizations. Key results include reduction in time spent on administrative tasks, decrease in payroll processing errors, and improvement in leave management efficiency. The centralized employee database has eliminated data redundancies and improved data accuracy. User surveys indicate satisfaction rate among HR personnel and employees, citing improved transparency and ease of access to information.

The EMS has successfully addressed the initial challenges of manual HR processes, providing a robust, scalable solution for workforce management. Its modular architecture has proven effective in adapting to diverse organizational needs. The system's reporting and analytics capabilities have enabled data-driven decision-making, leading to more strategic HR initiatives. To summarize, the EMS has significantly streamlined HR operations, enhanced employee satisfaction, and contributed to overall organizational efficiency, validating its effectiveness as a comprehensive HR management tool.

CHAPTER 6

CONCLUSION AND FUTURE WORK

While the Employee Management System has successfully addressed many HR management challenges, several areas for future development and enhancement have been identified:

1. AI-powered predictive analytics for workforce planning and talent management.

2. Integration with external job portals for streamlined recruitment processes.

3. Implementation of blockchain technology for secure, immutable record-keeping.

4. Development of a mobile application for improved accessibility and real-time updates.

5. Implementation of chatbot technology for instant employee query resolution.

6. Integration with IoT devices for advanced attendance and workspace utilization tracking.

These enhancements aim to further automate HR processes, improve user experience, and leverage cutting-edge technologies to keep the EMS at the forefront of HR management solutions.

APPENDIX 1

CODING

App.jsx :

```
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import AdminHome from './components/AdminHome';
import EmpHome from './components/EmpHome';
import Login from './components/Login';

function App() {
 return (
  <Router>
   <Routes>
    <Route path="/" element={<Login />} />
    <Route path="/adminhome" element={<AdminHome />} />
    <Route path="/emphome/:id" element={<EmpHome />} />
   </Routes>
  </Router>
 );
}

export default App;
```

AdminHome.jsx :

```
import React, { useState, useEffect } from 'react';
```

```
import axios from 'axios';
import { toast } from 'react-toastify';
import dayjs from 'dayjs';

import EmployeeList from './EmployeeList';
import AttendanceTracker from './AttendanceTracker';
import SalaryCalculator from './SalaryCalculator';
import SendCommonMessage from './SendCommonMessage';

export default function Component() {
  const [activeTab, setActiveTab] = useState('employees');
  const [holidays, setHolidays] = useState([]);
  const [newHoliday, setNewHoliday] = useState({ date: '', description: '' });
  const [employees, setEmployees] = useState([]);
  const [filteredEmployees, setFilteredEmployees] = useState([]);
  const [searchTerm, setSearchTerm] = useState('');
  const [newEmployee, setNewEmployee] = useState({
    name: '',
    email: '',
    department: '',
    salary: '',
    phoneNumber: '',
    sex: '',
    qualifications: '',
    role: '',
    dob: '',
    joiningDate: dayjs().format('YYYY-MM-DD'),
    experience: 'No Experience',
    experiencedRole: '',
    pf: '',
    da: ''
```

```
});
const [selectedEmployee, setSelectedEmployee] = useState(null);
const [leaves, setLeaves] = useState([]);
const [currentPage, setCurrentPage] = useState(1);
const [showAddEmployeeForm, setShowAddEmployeeForm] = useState(false);
const employeesPerPage = 4;
const [resignForm, setResignForm] = useState({ employeeId: '', resignationDate: '' });
const [salaryRecords, setSalaryRecords] = useState([]);

useEffect(() => {
  fetchEmployees();
  fetchLeaveRequests();
  fetchSalaryRecords();
  fetchHolidays();
}, []);

const fetchEmployees = async () => {
  try {
    const response = await axios.get('http://localhost:5000/employees');
    setEmployees(response.data);
    setFilteredEmployees(response.data);
  } catch (error) {
    toast.error('Failed to fetch employees');
  }
};

const fetchSalaryRecords = async () => {
  try {
    const response = await axios.get('http://localhost:5000/salary/calculate');
    setSalaryRecords(response.data);
  } catch (error) {
```

```
    toast.error('Failed to fetch salary records');
  }
};

const fetchHolidays = async () => {
  try {
    const response = await axios.get('http://localhost:5000/holidays');
    setHolidays(response.data);
  } catch (error) {
    toast.error('Failed to fetch holidays');
  }
};

const fetchLeaveRequests = async () => {
  try {
    const response = await axios.get('http://localhost:5000/leaves');
    setLeaves(response.data);
  } catch (error) {
    toast.error('Failed to fetch leave requests');
  }
};

const handleResignEmployee = async (e) => {
  e.preventDefault();
  try {
    await axios.put(`http://localhost:5000/employees/resign/${resignForm.employeeId}`, {
      resignationDate: resignForm.resignationDate
    });
    toast.success('Employee resigned successfully');
    fetchEmployees();
    setResignForm({ employeeId: '', resignationDate: '' });
```

```
    } catch (error) {
      toast.error('Failed to resign employee');
    }
  };

  const handleInputChange = (e) => {
    const { name, value } = e.target;
    setNewEmployee((prev) => ({ ...prev, [name]: value }));
  };

  const handleAddEmployee = async () => {
    try {
      const response = await axios.post('http://localhost:5000/employees', newEmployee);
      setEmployees((prev) => [...prev, response.data]);
      setFilteredEmployees((prev) => [...prev, response.data]);
      toast.success('Employee added successfully');
      resetEmployeeForm();
    } catch (error) {
      toast.error('Failed to add employee: ' + error.message);
    }
  };

  const handleUpdateEmployee = async () => {
    if (!selectedEmployee) return;
    try {
      const response = await
axios.put(`http://localhost:5000/employees/${selectedEmployee._id}`, newEmployee);
      setEmployees((prev) =>
        prev.map((employee) =>
          employee._id === selectedEmployee._id ? response.data : employee
        )
```

```
    );
    toast.success('Employee updated successfully');
    resetEmployeeForm();
  } catch (error) {
    toast.error('Failed to update employee');
  }
};

const resetEmployeeForm = () => {
  setSelectedEmployee(null);
  setNewEmployee({
    name: '',
    email: '',
    department: '',
    salary: '',
    phoneNumber: '',
    sex: '',
    qualifications: '',
    role: '',
    dob: '',
    joiningDate: dayjs().format('YYYY-MM-DD'),
    experience: 'No Experience',
    experiencedRole: '',
    pf: '',
    da: ''
  });
  setShowAddEmployeeForm(false);
};

const handleEditEmployee = (employee) => {
  setSelectedEmployee(employee);
```

```
  setNewEmployee(employee);
  setShowAddEmployeeForm(true);
};

// const handleDeleteEmployee = async (id) => {
//   try {
//     await axios.delete(`http://localhost:5000/employees/${id}`);
//     setEmployees((prev) => prev.filter((employee) => employee._id !== id));
//     setFilteredEmployees((prev) => prev.filter((employee) => employee._id !== id));
//     toast.success('Employee deleted successfully');
//   } catch (error) {
//     toast.error('Failed to delete employee');
//   }
// };

const handleDeleteEmployee = async (id) => {
  alert("You cannot delete an employee!! Instead resign him..");
};

const handleSearch = (e) => {
  const search = e.target.value.toLowerCase();
  setSearchTerm(search);
  const filtered = employees.filter((employee) =>
    employee.name.toLowerCase().includes(search) ||
    employee.employeeId.toLowerCase().includes(search) ||
    employee.department.toLowerCase().includes(search) ||
    employee.role.toLowerCase().includes(search)
  );
  setFilteredEmployees(filtered);
  setCurrentPage(1);
};
```

```
  const handleLeaveApproval = async (leaveId, newStatus) => {
    try {
      await axios.put(`http://localhost:5000/leaves/${leaveId}`, { status: newStatus });
      fetchLeaveRequests();
      toast.success(`Leave request ${newStatus.toLowerCase()} and email sent to employee`);
    } catch (error) {
      toast.error('Failed to update leave request');
    }
  };

  const handleAddHoliday = async (e) => {
    e.preventDefault();
    try {
      await axios.post('http://localhost:5000/holidays', newHoliday);
      toast.success('Holiday added successfully');
      fetchHolidays();
      setNewHoliday({ date: '', description: '' });
    } catch (error) {
      toast.error('Failed to add holiday');
    }
  };

  const totalPages = Math.ceil(filteredEmployees.length / employeesPerPage);
  const indexOfLastEmployee = currentPage * employeesPerPage;
  const indexOfFirstEmployee = indexOfLastEmployee - employeesPerPage;
  const currentEmployees = filteredEmployees.slice(indexOfFirstEmployee,
indexOfLastEmployee);

  return (
    <div className="admin-home">
```

```
    <h1 style={{textAlign:'center'}}>Employee Management System - Admin</h1>

    <div className="tabs" >
      <button className={activeTab === 'employees' ? 'active' : ''} onClick={() =>
setActiveTab('employees')}>Employees</button>
      <button className={activeTab === 'attendance' ? 'active' : ''} onClick={() =>
setActiveTab('attendance')}>Attendance</button>
      <button className={activeTab === 'leaves' ? 'active' : ''} onClick={() =>
setActiveTab('leaves')}>Leaves</button>
      <button className={activeTab === 'salary' ? 'active' : ''} onClick={() =>
setActiveTab('salary')}>Salary</button>
      <button className={activeTab === 'holidays' ? 'active' : ''} onClick={() =>
setActiveTab('holidays')}>Holidays</button>
      <button className={activeTab === 'resign' ? 'active' : ''} onClick={() =>
setActiveTab('resign')}>Resign Employee</button>
      <button className={activeTab === 'message' ? 'active' : ''} onClick={() =>
setActiveTab('message')}>Send Message</button>
    </div>

    {activeTab === 'employees' && (
      <div className="tab-content">
        <div className="search-add">
          <input style={{width:'10%'}}
            type="text"
            placeholder="Search employees"
            value={searchTerm}
            onChange={handleSearch}
          />
          <br/>
          <button onClick={() => setShowAddEmployeeForm(!showAddEmployeeForm)}>
            {showAddEmployeeForm ? 'Hide Form' : 'Add Employee'}
```

```
          </button>
        </div>

        {showAddEmployeeForm && (
          <div className="add-employee-form">
            <h3>{selectedEmployee ? 'Update Employee' : 'Add New Employee'}</h3>
            <div className="form-grid">
              <input name="name" placeholder="Name" value={newEmployee.name}
onChange={handleInputChange} />
              <input name="email" type="email" placeholder="Email"
value={newEmployee.email} onChange={handleInputChange} />
              <input name="department" placeholder="Department"
value={newEmployee.department} onChange={handleInputChange} />
              <input name="salary" type="number" placeholder="Salary"
value={newEmployee.salary} onChange={handleInputChange} />
              <input name="pf" type="number" placeholder="PF" value={newEmployee.pf}
onChange={handleInputChange} />
              <input name="da" type="number" placeholder="DA" value={newEmployee.da}
onChange={handleInputChange} />
              <input name="phoneNumber" placeholder="Phone Number"
value={newEmployee.phoneNumber} onChange={handleInputChange} />
              <input name="sex" placeholder="Sex" value={newEmployee.sex}
onChange={handleInputChange} />
              <input name="qualifications" placeholder="Qualifications"
value={newEmployee.qualifications} onChange={handleInputChange} />
              <input name="role" placeholder="Role" value={newEmployee.role}
onChange={handleInputChange} />
              <input name="dob" type="date" placeholder="Date of Birth"
value={newEmployee.dob} onChange={handleInputChange} />
              <input name="joiningDate" type="date" placeholder="Joining Date"
value={newEmployee.joiningDate} disabled />
```

```
            <select name="experience" value={newEmployee.experience}
onChange={handleInputChange}>
              <option value="No Experience">No Experience</option>
              <option value="1 Year">1 Year</option>
              <option value="2 Years">2 Years</option>
              <option value="3 Years">3 Years</option>
              <option value="4 Years">4 Years</option>
              <option value="5+ Years">5+ Years</option>
            </select>
            {newEmployee.experience !== 'No Experience' && (
              <input
                name="experiencedRole"
                placeholder="Experienced Role"
                value={newEmployee.experiencedRole}
                onChange={handleInputChange}
              />
            )}
          </div>
          <button onClick={selectedEmployee ? handleUpdateEmployee :
handleAddEmployee}>
            {selectedEmployee ? 'Update Employee' : 'Add Employee'}
          </button>
        </div>
      )}

      <EmployeeList
        employees={currentEmployees}
        onEdit={handleEditEmployee}
        onDelete={handleDeleteEmployee}
        onResign={handleResignEmployee}
      />
```

```
        <div className="pagination"
style={{textAlign:'center',marginTop:'50px',padding:'10px'}}>
          <button style={{margin:'10px'}} onClick={() => setCurrentPage(prev =>
Math.max(prev - 1, 1))} disabled={currentPage === 1}>
            Previous
          </button>
          <span>Page {currentPage} of {totalPages}</span>
          <button style={{margin:'10px'}} onClick={() => setCurrentPage(prev =>
Math.min(prev + 1, totalPages))} disabled={currentPage === totalPages}>
            Next
          </button>
        </div>
      </div>
    )}

    {activeTab === 'attendance' && (
      <div className="tab-content">
        <h2>Attendance Management</h2>
        <AttendanceTracker />
      </div>
    )}

    {activeTab === 'leaves' && (
      <div className="tab-content">
        <h2>Leave Requests</h2>
        <table className="leave-requests">
          <thead>
            <tr>
              <th>Employee</th>
              <th>Start Date</th>
```

```
            <th>End Date</th>
            <th>Reason</th>
            <th>Status</th>
            <th>Actions</th>
          </tr>
        </thead>
        <tbody>
          {leaves.map((leave) => (
            <tr key={leave._id}>
              <td>{leave.employeeName} ({leave.employeeId})</td>
              <td>{leave.startDate}</td>
              <td>{leave.endDate}</td>
              <td>{leave.reason}</td>
              <td>{leave.status}</td>
              <td>
                {leave.status === 'Pending' && (
                  <>
                    <button onClick={() => handleLeaveApproval(leave._id,
'Approved')}>Approve</button>
                    <button onClick={() => handleLeaveApproval(leave._id, 'Rejected')}
className="reject">Reject</button>
                  </>
                )}
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  )}
```

```
{activeTab === 'salary' && (
 <div className="tab-content">
  <h2>Salary Management</h2>
  <table className="salary-table">
   <thead>
    <tr>
     <th>Employee ID</th>
     <th>Name</th>
     <th>Present Days</th>
     <th>Total Days</th>
     <th>Working Days</th>
     <th>Base Salary (₹)</th>
     <th>Holiday Pay (₹)</th>
     <th>DA (₹)</th>
     <th>PF (₹)</th>
     <th>Deductions (₹)</th>
     <th>Net Salary (₹)</th>
     <th>Uninformed Leaves</th>

    </tr>
   </thead>
   <tbody>
    {salaryRecords.map((record) => (
     <tr key={record.employeeId}>
      <td>{record.employeeId}</td>
      <td>{record.name}</td>
      <td>{record.presentDays}</td>
      <td>{record.totalDays}</td>
      <td>{record.workingDays}</td>
      <td>₹{record.baseSalary.toLocaleString()}</td>
      <td>₹{record.holidayPay.toLocaleString()}</td>
```

```
            <td>₹{record.da.toLocaleString()}</td>
            <td>₹{record.pf.toLocaleString()}</td>
            <td>₹{record.deductions.toLocaleString()}</td>
            <td>₹{record.netSalary.toLocaleString()}</td>
            <td>{record.uninformedLeaveDays}</td>

          </tr>
        ))}
      </tbody>
    </table>
  </div>
)}

{activeTab === 'holidays' && (
  <div className="tab-content">
    <h2>Holiday Management</h2>
    <form onSubmit={handleAddHoliday} className="add-holiday-form">
    <input style={{width:'30%',margin:'10px'}}
      type="date"
      value={newHoliday.date}
      onChange={(e) => setNewHoliday({ ...newHoliday, date: e.target.value })}
      required
     />
     <input style={{width:'30%',margin:'10px'}}
      type="text"
      value={newHoliday.description}
      onChange={(e) => setNewHoliday({ ...newHoliday, description: e.target.value })}
      placeholder="Holiday Description"
      required
     />
     <button type="submit">Add Holiday</button>
```

```
    </form>

    <table className="holidays-table">
      <thead>
        <tr>
          <th>Date</th>
          <th>Description</th>
        </tr>
      </thead>
      <tbody>
        {holidays.map((holiday) => (
          <tr key={holiday._id}>
            <td>{holiday.date}</td>
            <td>{holiday.description}</td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
)}

{activeTab === 'resign' && (
  <div className="tab-content">
    <h2>Resign Employee</h2>
    <form onSubmit={handleResignEmployee} className="resign-form">
      <div>
        <label htmlFor="employeeId">Employee ID:</label>
        <input style={{width:'30%',margin:'10px'}}
          type="text"
          id="employeeId"
          value={resignForm.employeeId}
```

```
            onChange={(e) => setResignForm({ ...resignForm, employeeId: e.target.value })}
            required
           />
          </div>
          <div>
           <label htmlFor="resignationDate">Resignation Date:</label>
           <input style={{width:'30%',margin:'10px'}}
            type="date"
            id="resignationDate"
            value={resignForm.resignationDate}
            onChange={(e) => setResignForm({ ...resignForm, resignationDate: e.target.value
})}
            required
           />
          </div>
          <button type="submit">Submit Resignation</button>
         </form>
        </div>
      )}

      {activeTab === 'message' && (
       <div className="tab-content">
        <h2>Send Common Message</h2>
        <SendCommonMessage />
       </div>
      )}
    </div>
  );
}
export default BasicInfoSection;
```

EmpHome.js :

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { useParams } from 'react-router-dom';
import dayjs from 'dayjs';
import './EmpHome.css';

export default function Component() {
  const { id } = useParams();

  const [employee, setEmployee] = useState(null);
  const [leaveForm, setLeaveForm] = useState({
    reason: '',
    startDate: dayjs().format('YYYY-MM-DD'),
    endDate: dayjs().format('YYYY-MM-DD')
  });
  const [leaves, setLeaves] = useState([]);
  const [experience, setExperience] = useState('');
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchData = async () => {
      try {
        setLoading(true);
        const employeeResponse = await
axios.get(`http://localhost:5000/employees/search/${id}`);
        setEmployee(employeeResponse.data);
        if (employeeResponse.data && employeeResponse.data.joiningDate) {
          calculateExperience(employeeResponse.data.joiningDate);
```

```
      }
      if (!employeeResponse.data.resignationDate) {
        const leavesResponse = await
axios.get(`http://localhost:5000/leaves/employee/${id}`);
        setLeaves(leavesResponse.data);
      }
    } catch (error) {
      setError('Failed to fetch employee data. Please try again later.');
      console.error('Error fetching employee data:', error);
    } finally {
      setLoading(false);
    }
  };

  fetchData();
}, [id]);

const calculateExperience = (joiningDate) => {
  const today = dayjs();
  const joining = dayjs(joiningDate);
  const years = today.diff(joining, 'year');
  setExperience(`${years} year(s)`);
};

const handleLeaveApplication = async () => {
  try {
    if (!employee || employee.resignationDate) {
      throw new Error('Cannot apply for leave');
    }
    await axios.post('http://localhost:5000/leaves', {
      employeeId: id,
```

```
      employeeName: employee.name,
      ...leaveForm
    });
    const response = await axios.get(`http://localhost:5000/leaves/employee/${id}`);
    setLeaves(response.data);
    setLeaveForm({ reason: '', startDate: dayjs().format('YYYY-MM-DD'), endDate:
dayjs().format('YYYY-MM-DD') });
  } catch (error) {
    setError('Failed to submit leave request.');
    console.error('Error submitting leave request:', error);
  }
};

const getStatusColor = (status) => {
  switch (status) {
    case 'Approved': return 'text-green-600';
    case 'Rejected': return 'text-red-600';
    case 'Pending': return 'text-yellow-600';
    default: return 'text-gray-600';
  }
};

if (loading) {
  return <div className="loading">Loading...</div>;
}

if (error) {
  return <div className="error" role="alert">{error}</div>;
}

if (!employee) {
```

```
  return <div className="not-found" role="alert">Employee not found.</div>;
 }

 return (
  <div className="max-w-4xl mx-auto p-6">
   <div className="bg-white rounded-lg shadow-lg p-6 mb-6">
    <h2 className="text-2xl font-bold mb-4">Welcome, {employee.name}</h2>

    {/* Employee Details Section */}
    <div className="bg-gray-50 p-4 rounded-lg mb-6">
     <h3 className="text-xl font-semibold mb-4">Employee Details</h3>
     <div className="grid grid-cols-2 gap-4">
      <p><span className="font-semibold">ID:</span> {employee.employeeId}</p>
      <p><span className="font-semibold">Name:</span> {employee.name}</p>
      <p><span className="font-semibold">Email:</span> {employee.email}</p>
      <p><span className="font-semibold">Department:</span>
{employee.department}</p>
      <p><span className="font-semibold">Role:</span> {employee.role}</p>
      <p><span className="font-semibold">Phone:</span>
{employee.phoneNumber}</p>
      <p><span className="font-semibold">Sex:</span> {employee.sex}</p>
      <p><span className="font-semibold">Date of Birth:</span> {employee.dob}</p>
      <p><span className="font-semibold">Joining Date:</span>
{employee.joiningDate}</p>
      <p><span className="font-semibold">Experience:</span> {experience}</p>
      {employee.resignationDate && (
       <p className="col-span-2 text-red-600">Resigned on:
{employee.resignationDate}</p>
      )}
     </div>
    </div>
```

```
{!employee.resignationDate && (
 <>
  {/* Leave Application Section */}
  <div className="mb-8">
  <h3 className="text-xl font-semibold mb-4">Apply for Leave</h3>
  <div className="space-y-4">
   <input
    type="text"
    placeholder="Reason for leave"
    value={leaveForm.reason}
    onChange={(e) => setLeaveForm(prev => ({ ...prev, reason: e.target.value }))}
    className="w-full p-2 border rounded"
   />
   <div className="flex space-x-4">
    <input
     type="date"
     value={leaveForm.startDate}
     onChange={(e) => setLeaveForm(prev => ({ ...prev, startDate: e.target.value }))}
     className="w-1/2 p-2 border rounded"
    />
    <input
     type="date"
     value={leaveForm.endDate}
     onChange={(e) => setLeaveForm(prev => ({ ...prev, endDate: e.target.value }))}
     className="w-1/2 p-2 border rounded"
    />
   </div>
   <button
    onClick={handleLeaveApplication}
    className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700"
```

```
    >
      Submit Leave Request
    </button>
  </div>
</div>

  {/* Attendance Section */}
  <div className="attendance-section mb-8">
    <h3 className="text-xl font-semibold mb-4">Attendance</h3>
    {employee.attendance && employee.attendance.length > 0 ? (
      <table className="w-full">
        <thead>
          <tr>
            <th className="border p-2">Date</th>
            <th className="border p-2">Status</th>
          </tr>
        </thead>
        <tbody>
          {employee.attendance.map((record, index) => (
            <tr key={index}>
              <td className="border p-2">{record.date}</td>
              <td className="border p-2">{record.status}</td>
            </tr>
          ))}
        </tbody>
      </table>
    ) : (
      <p>No attendance records available.</p>
    )}
  </div>
```

```
        {/* Salary Section */}
        {employee.monthlySalary && (
          <div className="salary-section mb-8">
            <h3 className="text-xl font-semibold mb-4">Current Month Salary Details</h3>
            <p><span className="font-semibold">Month:</span>
{employee.monthlySalary.month}</p>
            <p><span className="font-semibold">Present Days:</span>
{employee.monthlySalary.presentDays}</p>
            {/* <p><span className="font-semibold">Total Working Days:</span>
{employee.monthlySalary.workingDays}</p> */}
            <p><span className="font-semibold">Base Salary:</span>
₹{employee.salary?.toLocaleString() ?? 'N/A'}</p>
            {/* <p><span className="font-semibold">Holiday Pay:</span>
₹{employee.monthlySalary.holidayPay?.toLocaleString() ?? 'N/A'}</p> */}
            <p><span className="font-semibold">DA:</span>
₹{employee.da?.toLocaleString() ?? 'N/A'}</p>
            <p><span className="font-semibold">PF:</span>
₹{employee.pf?.toLocaleString() ?? 'N/A'}</p>
            {/* <p><span className="font-semibold">Deductions:</span>
₹{employee.monthlySalary.deductions?.toLocaleString() ?? 'N/A'}</p> */}
            <p><span className="font-semibold">Net Salary:</span>
₹{employee.monthlySalary.netSalary?.toLocaleString() ?? 'N/A'}</p>
          </div>
        )}

        {/* Leave Requests Section */}
        <div>
          <h3 className="text-xl font-semibold mb-4">Leave Requests</h3>

          {leaves.length > 0 ? (
            <div className="space-y-4">
```

```
                {leaves.map((leave) => (
                  <div key={leave._id} className="bg-gray-50 p-3 rounded">
                    <p><span className="font-medium">Date:</span> {leave.date}</p>
                    <p><span className="font-medium">Reason:</span> {leave.reason}</p>
                    <p className={getStatusColor(leave.status)}>Status: {leave.status}</p>
                  </div>
                ))}
              </div>
            ) : (
              <p className="text-gray-500">No leave requests found</p>
            )}
          </div>
        </>
      )}
    </div>
  </div>
 );
}
```

Login.js :

```
import React, { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { Form, Input, Button, Checkbox } from 'antd';
import '../App.css';
import './Login.css';
import photo from '../assets/6300830.jpg';

const Login = () => {
  const [credentials, setCredentials] = useState({ username: '', password: '' });
```

```
const navigate = useNavigate();

const onFinish = (values) => {
  const { username, password } = values;
  setCredentials({ username, password });

  if (username === 'admin' && password === 'admin') {
    navigate('/adminhome');
  } else if (password === 'emp') {
    navigate(`/emphome/${username}`);
  } else {
    alert('Invalid credentials');
  }
};

return (
  <div className="login-page">
    <div className="login-box">
      <div className="illustration-wrapper">
        <img
          src={photo}
          alt="Login"
        />
      </div>
      <Form
        name="login-form"
        initialValues={{ remember: true }}
        onFinish={onFinish}
      >
        <p className="form-title">Welcome back</p>
        <p>Login to the Dashboard</p>
```

```
        <Form.Item
          name="username"
          rules={[{ required: true, message: 'Please input your username!' }]}
        >
          <Input placeholder="Username" />
        </Form.Item>

        <Form.Item
          name="password"
          rules={[{ required: true, message: 'Please input your password!' }]}
        >
          <Input.Password placeholder="Password" />
        </Form.Item>

        <Form.Item>
          <Button type="primary" htmlType="submit" className="login-form-button">
            LOGIN
          </Button>
        </Form.Item>
      </Form>
    </div>
  </div>
 );
};

export default Login;
```

SalaryCalculator.jsx :

```
 import React, { useState } from 'react';
```

```
import axios from 'axios';
import './SalaryCalculator.css';
const SalaryCalculator = () => {
  const [salaryRecords, setSalaryRecords] = useState([]);
  const [error, setError] = useState('');

  const calculateSalary = async () => {
    try {
      const response = await axios.get('http://localhost:5000/salary/calculate');
      setSalaryRecords(response.data);
      setError('');
      alert('Salaries calculated successfully');
    } catch (error) {
      console.error('Error calculating salaries:', error);
      setError('Failed to calculate salaries. Please try again.');
    }
  };

  return (
    <div className="salary-calculator">
      <h2>Salary Calculator</h2>
      {error && <div className="error">{error}</div>}
      <button onClick={calculateSalary}>Calculate Salaries</button>
      {salaryRecords.length > 0 && (
        <div className="salary-records">
          <h3>Salary Records</h3>
          <table>
            <thead>
              <tr>
                <th>Employee ID</th>
                <th>Name</th>
```

```
            <th>Present Days</th>
            <th>Total Days</th>
            <th>Base Salary (₹)</th>
            <th>DA (₹)</th>
            <th>PF (₹)</th>
            <th>Net Salary (₹)</th>
          </tr>
        </thead>
        <tbody>
          {salaryRecords.map((record) => (
            <tr key={record.employeeId}>
              <td>{record.employeeId}</td>
              <td>{record.name}</td>
              <td>{record.presentDays}</td>
              <td>{record.totalDays}</td>
              <td>₹{record.baseSalary.toLocaleString()}</td>
              <td>₹{record.da.toLocaleString()}</td>
              <td>₹{record.pf.toLocaleString()}</td>
              <td>₹{record.netSalary.toLocaleString()}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  )}
 </div>
 );
};

export default SalaryCalculator;
```

APPENDIX 2

SNAPSHOTS

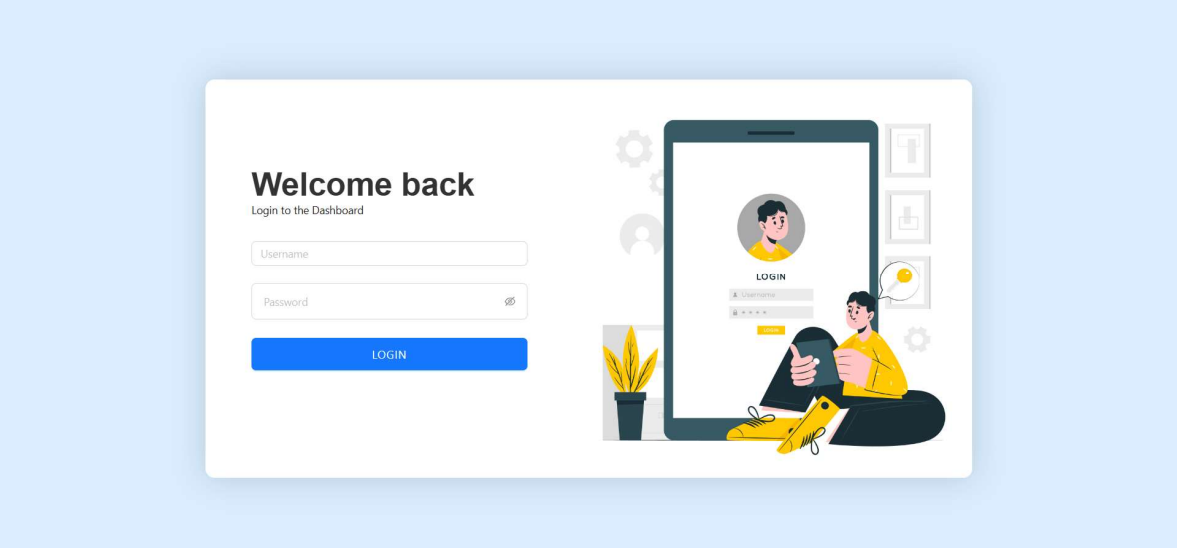


Figure A2.1 LandingPage

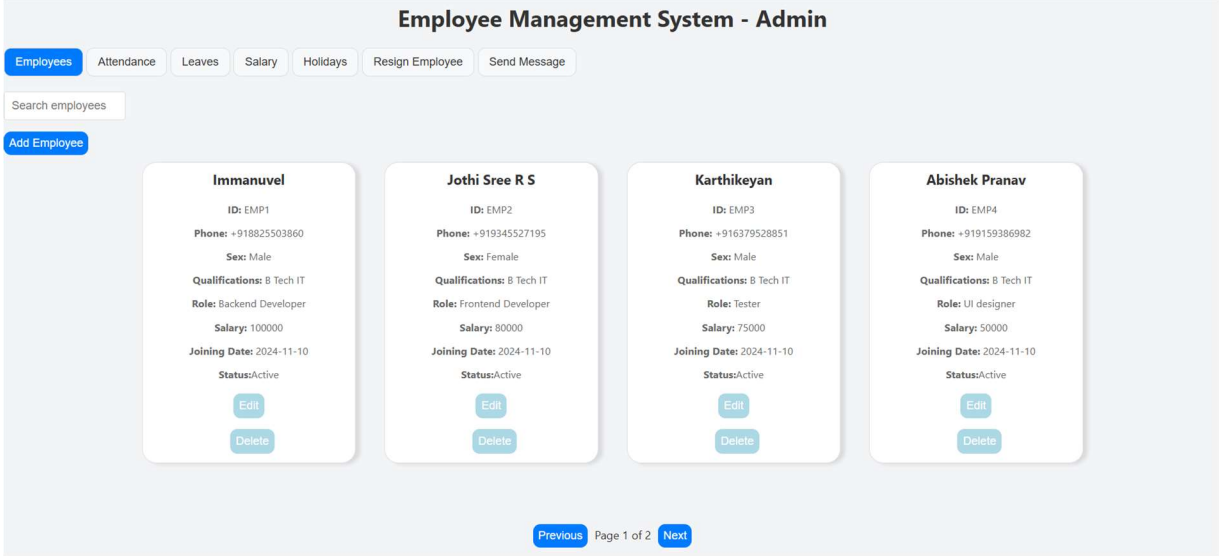


Figure A2.2 Admin Panel

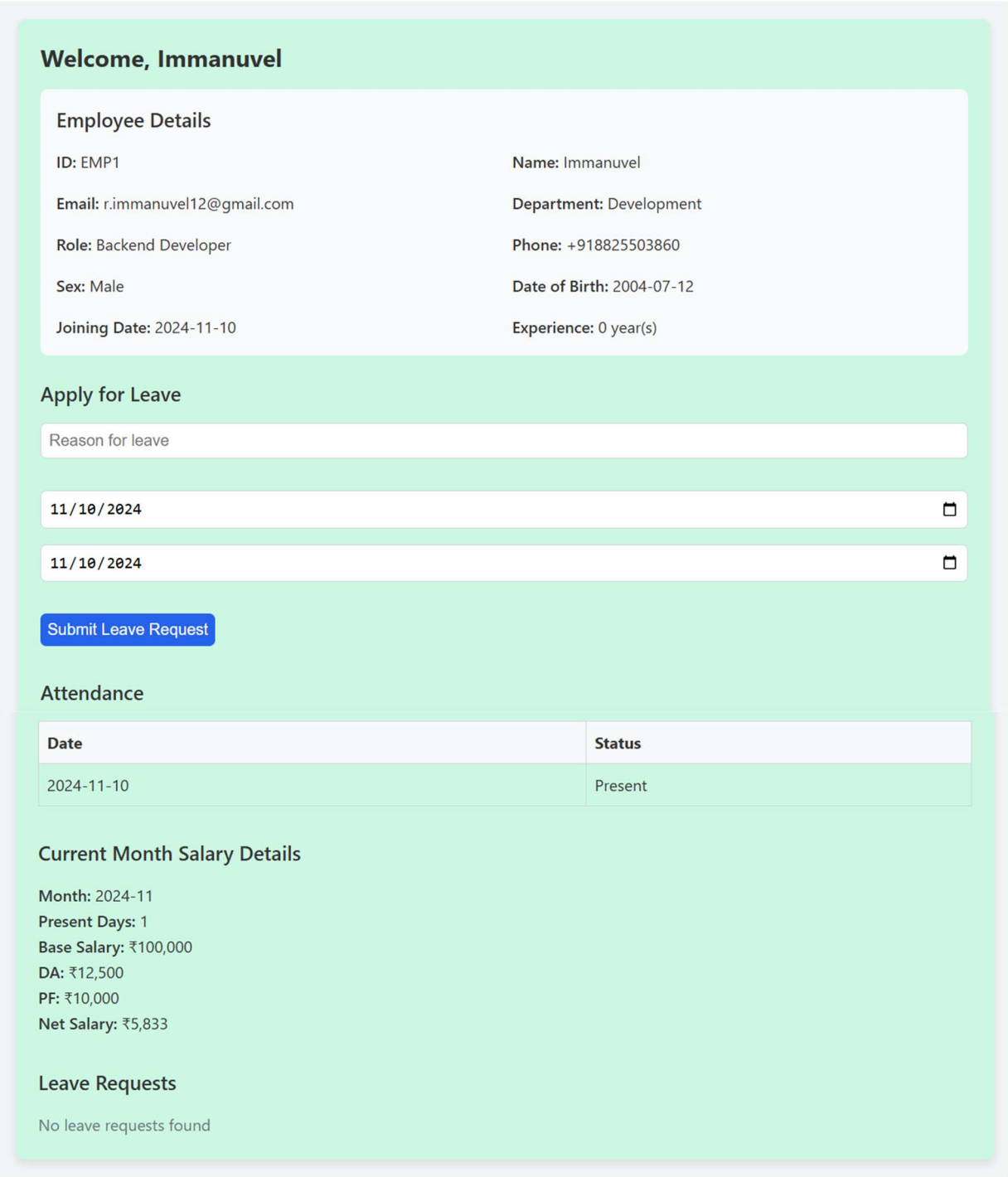


Figure A2.3 Employee Panel

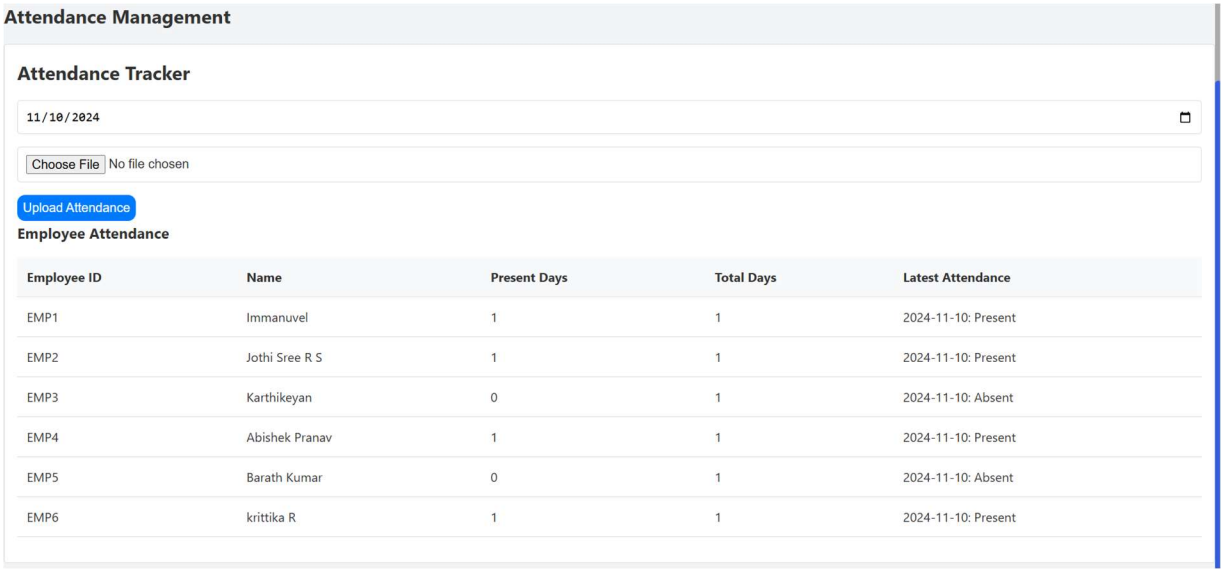


Figure A2.4 Attendance Page

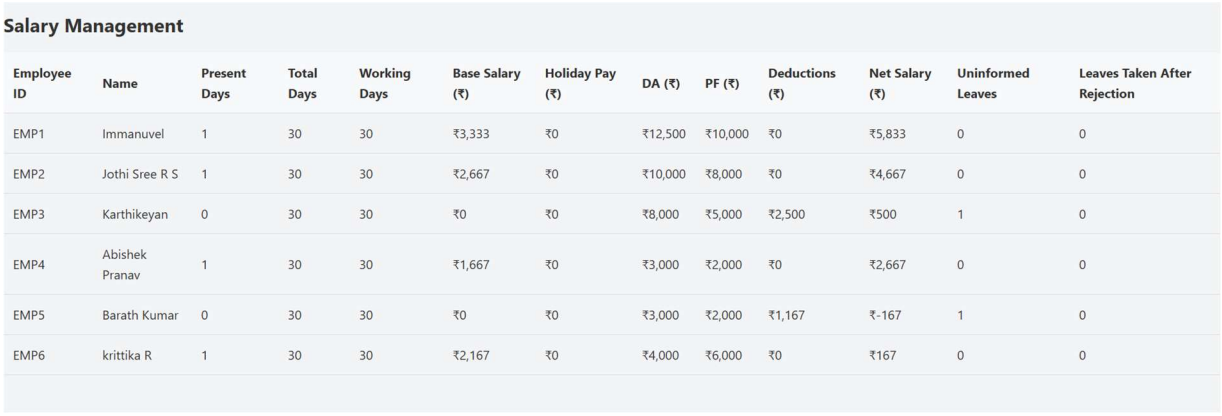


Figure A2.5 Salary Page



Figure A2.6 Leave Page

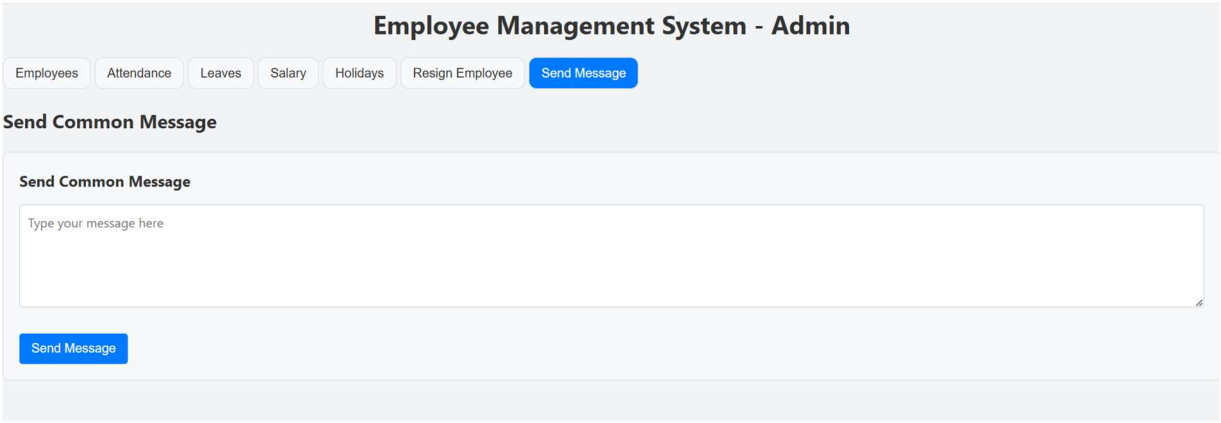


Figure A2.7 Common message Page

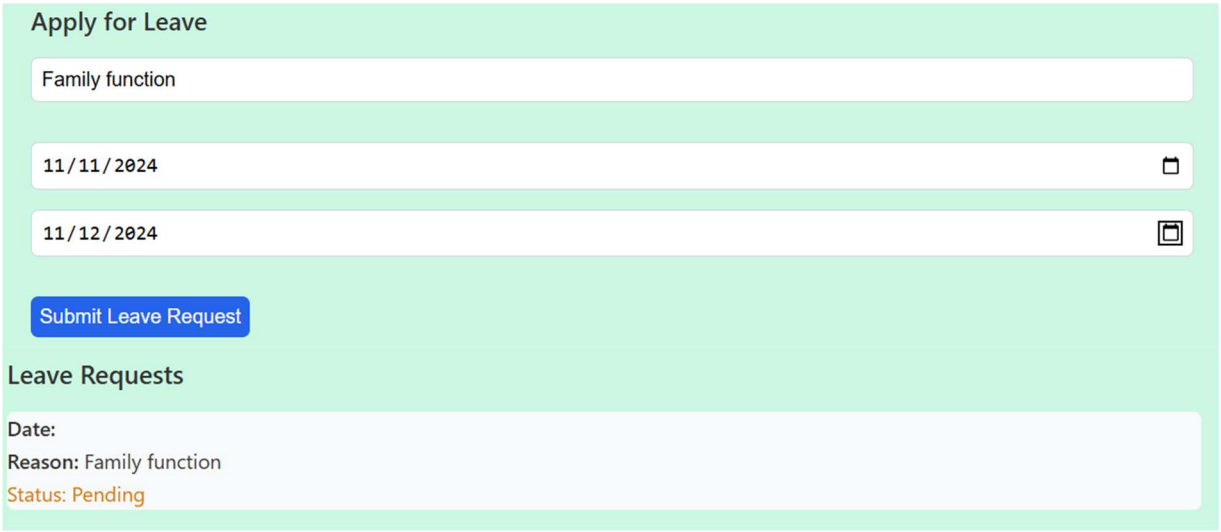


Figure A2.8 Leave application Page

REFERENCES

GITHUB LINK: **https://github.com/Immanuvel1207/Employee-Management-System**

https://expressjs.com/en/guide/using-middleware.html

https://www.mongodb.com/developer/languages/javascript/getting-started-with-mongodb- and-mongoose/

https://www.twilio.com/docs#twilio-essentials

https://www.nodemailer.com/

https://react.dev/